HybridACD: Adversarial Constraint Decoding for Logically Consistent Large Language Model Forecasting

Kiet Tran Anh[1*]

[1*]Faculty of Information Science and Engineering, University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam.

Corresponding author(s). E-mail(s): 23520820@gm.uit.edu.vn;

Abstract

This report investigates the problem of *probabilistic consistency* in large language models (LLMs) when used as forecasting agents. Although LLMs can produce reasonable probability estimates at the level of individual questions, their predictions may still violate fundamental logical relations such as negation, paraphrase, conjunction, disjunction, and conditional probability. Such violations undermine the reliability of a forecasting system and can be interpreted as Dutch Book vulnerabilities in probability space.

To address this problem, we propose **HybridACD** (*Hybrid Adversarial Consistency Decoding*), a decoding-time intervention architecture that combines two mechanisms: (i) adversarial rewriting to generate question variants that are semantically equivalent but resistant to surface pattern matching, and (ii) token-constrained decoding to bound the output probability within a valid interval derived from the consistency rules. Unlike costly post-hoc calibration or model fine-tuning that risks Goodhart's law, HybridACD leaves model weights untouched and imposes constraints only at the final probability-extraction step. Experiments on six LLMs show that HybridACD reduces the mean Aggregate Violation Score from **0.1522** to **0.0053** (**−95.5%**), while improving the Brier Score by **2.4%** to **17.3%** on the models evaluated in full. The results suggest that decoding-time constraints are an effective approach for enhancing the probabilistic consistency of LLM forecasters without degrading forecasting accuracy.

Contents

1 Introduction

Large language models (LLMs) are increasingly used for probabilistic forecasting tasks, where the model must assign a value $p \in [0, 1]$ to the likelihood of an event. Unlike ordinary classification problems, forecasting requires the model not only to produce a plausible single answer but also to maintain a consistent belief system across multiple logically related questions. For example, if a model predicts $P(A) = 0.70$, then the probability of the negated event $\neg A$ must be 0.30. Otherwise, the system violates the axiom of additivity.

Recent studies on LLM forecasters show that consistency scores can serve as an early evaluation signal even before ground truth is available [1]. This is particularly important for future-oriented forecasting problems, where waiting for the actual outcome may take a long time. However, good calibration on individual questions does not guarantee that a model is consistent across groups of logically related questions. An LLM may achieve a low Brier Score while still producing contradictory probabilities for negated events, equivalent events, or compound events.

This problem can be formalized through the Dutch Book argument: if a set of probability predictions is inconsistent, an adversary can construct a betting portfolio that guarantees a positive profit regardless of the actual outcome. Probabilistic consistency is therefore not merely a formal requirement, but a necessary condition for an LLM forecaster to be regarded as a trustworthy decision-making system. Post-hoc methods such as arbitrage calibration can reduce violations, but typically require multiple rounds of model calls and can incur substantial cost. Conversely, optimizing model weights directly can trigger a Goodhart effect: the model learns to optimize the consistency metric via neutral predictions such as $P = 0.5$, which degrades real forecasting value.

This report focuses on **HybridACD**, a hybrid architecture that combines adversarial rewriting with token-constrained decoding. The core idea is to impose probabilistic constraints at decoding time, before the final probability is finalized. This approach offers three advantages: no model retraining is required, no iterative post-hoc calibration is needed, and probabilistic rules can be translated directly into valid intervals for the output tokens.

Main contributions of this report include:

1. Presenting a HybridACD framework for LLM forecasting that combines adversarial rewriting with decoding-time probability constraints.
2. Formalizing how probabilistic consistency rules are converted into bounds $[\ell, u]$ used in Token Constraint Decoding (TCD).
3. Empirically evaluating HybridACD across multiple LLMs, focusing on three criteria: reduction of consistency violations, improvement of the Brier Score, and control of inference cost.

2 Related Work

2.1 Probabilistic consistency in LLM forecasters

Paleka et al. proposed evaluating LLM forecasters through consistency checks across multiple logically related questions [1]. Rather than scoring a forecast only after the event occurs, this approach checks whether the probabilities generated by the model comply with relations such as negation, entailment, conjunction, disjunction, and conditional probability. This work is a direct foundation for HybridACD, since the goal of the system is not merely to forecast each question correctly but also to maintain a valid probability distribution over the entire question set.

At a basic level, an LLM generates text by predicting the next token from a conditional probability distribution [2]. However, the intrinsic token probability does not necessarily correspond to the logical soundness of the answer. Analyses of probabilistic consistency show that a model may favor token sequences that are natural in language but inconsistent in probability [3, 4]. This explains why a dedicated intervention mechanism at the output layer is needed, rather than relying solely on prompting or sequence probability.

2.2 Preference optimization and self-consistency

One direction for improving reasoning is to leverage intrinsic probability signals during optimization. Yang et al. proposed Probability-Consistent Preference Optimization (PCPO), in which preference selection is based not only on the correctness of the answer but also on token-level probabilistic consistency [5]. Zhou et al. linked intrinsic probability to self-consistency by using perplexity to select or discard unreliable reasoning paths [6]. These works show that intrinsic probability can support reasoning, but they largely operate at the training or sampling stage. HybridACD differs in that it imposes constraints directly at the point where the final probability is decoded.

2.3 Constrained decoding and decoding-time alignment

Constrained decoding is a family of methods that control LLM output by restricting the set of valid tokens during text generation. Koo et al. used automata to represent formal constraints, which are particularly useful for structured outputs such as JSON, API calls, or expressions with a defined grammar [7]. Yao et al. showed that Token Constraint Decoding can improve robustness in question-answering tasks where the output must belong to a valid token set [8]. Huang et al. extended this idea with DeAL, a decoding-time alignment framework that can combine multiple rewards and declarative constraints [9].

However, constrained decoding also has drawbacks. Schall and de Melo showed that structural constraints can degrade text generation performance, particularly for instruction-tuned models, because the model is pulled away from the natural language patterns it prefers [10]. For this reason, HybridACD applies constraints only at the numerical probability-extraction step rather than constraining the entire reasoning process. This design preserves the model's free-form reasoning ability while controlling the final probability value to satisfy the probability axioms.

2.4 Controlled prompting and adversarial robustness

Mousavi and Termehchy [11] showed that controlled prompting and decoding can help reduce inconsistent answers across equivalent queries. In HybridACD, the adversarial rewriting component serves a similar purpose but within the context of probabilistic forecasting: questions are transformed syntactically, with entity substitution or nested conditioning, while the resolution criterion is preserved. This limits the extent to which the model merely learns the surface patterns of the consistency test suite.

Taken together, these strands of research point to a gap in combining three elements: logical consistency evaluation, exploitation of intrinsic probability, and decoding-time output control. HybridACD is designed to fill this gap with a lightweight pipeline that requires no retraining and can be applied to a wide range of underlying LLMs.

3 Method

3.1 Problem statement

Let $\mathcal{Q}$ denote the set of binary forecasting questions. A forecaster $\mathcal{F}$ maps each question $Q \in \mathcal{Q}$ to a subjective probability:

$$\mathcal{F} : \mathcal{Q} \to [0, 1], \quad p = \mathcal{F}(Q). \tag{1}$$

A question set $T = \{Q_1, \ldots, Q_k\}$ is called a consistency set if there exists a logical relation $\mathcal{L}$ among the questions that implies a probabilistic constraint $\mathcal{C}(p_1, \ldots, p_k)$. For example, under the negation rule, $\mathcal{C}$ requires $p_P + p_{\neg P} = 1$.

The goal of HybridACD is to generate predictions $\hat{p}_1, \ldots, \hat{p}_k$ such that:

$$\mathcal{C}(\hat{p}_1, \ldots, \hat{p}_k) = 0 \quad \text{and} \quad \mathbb{E}[(\hat{p} - y)^2] \text{ is minimized as much as possible,} \tag{2}$$

where $(\hat{p} - y)^2$ is the Brier Score and $y \in \{0, 1\}$ is the actual outcome.

3.2 Consistency rules

HybridACD uses ten probabilistic consistency rules, summarized in Table 1. Each rule is converted into a feasible interval $[\ell, u]$ for the current prediction, based on previously obtained predictions.

Here, ℓ and u denote the lower and upper bounds imposed on the target variable given previously known values.

Table 1 Ten probabilistic consistency rules and their mathematical constraints

#	Rule	Constraint $\mathcal{C}(p_1, \ldots, p_k)$	Key Set
1	Negation	$p_P + p_{\neg P} = 1$	$\{P, \neg P\}$
2	Paraphrase	$p_P = p_{P'}$	$\{P, P'\}$
3	Entailment	$P \models B \Rightarrow p_P \leq p_B$	$\{P, B\}$
4	And	$\max(0, p + q - 1) \leq p_{P \wedge Q} \leq \min(p, q)$	$\{P, Q, P \wedge Q\}$
5	Or	$\max(p, q) \leq p_{P \vee Q} \leq \min(1, p + q)$	$\{P, Q, P \vee Q\}$
6	But	$p_{P \vee Q} = p_P + p_{Q \wedge \neg P}$	$\{P, Q \wedge \neg P, P \vee Q\}$
7	Conditional	$p_{P \wedge Q} = p_P \cdot p_{Q \mid P}$	$\{P, Q \mid P, P \wedge Q\}$
8	Nested Conditional	$p_{P \wedge Q \wedge R} = p_P \cdot p_{Q \mid P} \cdot p_{R \mid P \wedge Q}$	$\{P, Q \mid P, R \mid P \wedge Q, P \wedge Q \wedge R\}$
9	And-Or	$p_P + p_Q = p_{P \vee Q} + p_{P \wedge Q}$	$\{P, Q, P \wedge Q, P \vee Q\}$
10	Expected Evidence	$p_P = p_Q p_{P \mid Q} + (1 - p_Q) p_{P \mid \neg Q}$	$\{P, Q, P \mid Q, P \mid \neg Q\}$

3.3 Overview of the HybridACD architecture

HybridACD consists of two main components. The first is the *Adversarial Input Agent*, which uses an auxiliary language model to rewrite the forecasting question so that it is syntactically more complex while preserving the resolution criterion. The second is

Token Constraint Decoding, which computes the valid probability interval and forces the analyzing model to return a value within that interval.

For question Q_i, the adversarial agent generates a variant:

$$Q_i' = \mathcal{M}_{adv}(Q_i; \tau = 0.7), \quad \text{Sem}(Q_i') = \text{Sem}(Q_i). \tag{3}$$

Transformation types include inserting subordinate clauses, substituting entities, raising negation, and nesting conditions. The goal is not to change the label but to reduce the model's tendency to rely on pattern matching.

The main model then generates free-form reasoning:

$$\text{cot}_i = \mathcal{M}(Q_i'). \tag{4}$$

This reasoning step is unconstrained, allowing the model to still benefit from Chain-of-Thought. The constraint is applied only when the final probability is extracted.

3.4 Computing probability bounds

Suppose the previous predictions are $\hat{p}_1, \ldots, \hat{p}_{i-1}$. For logical rule $\mathcal{L}$, HybridACD computes:

$$\ell_i = \max(0, \phi_\ell(\hat{p}_1, \ldots, \hat{p}_{i-1}; \mathcal{L})), \quad u_i = \min(1, \phi_u(\hat{p}_1, \ldots, \hat{p}_{i-1}; \mathcal{L})). \tag{5}$$

For example, under the COND rule, if $\hat{p}_P$ and $\hat{p}_{Q|P}$ are already known, then the valid value for $P \wedge Q$ is:

$$\ell = u = \hat{p}_P \cdot \hat{p}_{Q|P}. \tag{6}$$

Under the AND rule, the valid value for $P \wedge Q$ lies within:

$$\max(0, \hat{p}_P + \hat{p}_Q - 1) \leq \hat{p}_{P \wedge Q} \leq \min(\hat{p}_P, \hat{p}_Q). \tag{7}$$

3.5 Token-constrained decoding

The TCD mechanism operates on a discrete set of probability values:

$$\mathcal{V}_{num} = \{0.00; 0.01; \ldots; 1.00\}.$$

Each value $v \in \mathcal{V}_{num}$ is tokenized to identify the token representing that probability. Given the valid interval $[\ell_i, u_i]$, HybridACD constructs a logit bias:

$$b_{t_v} = \begin{cases} 0, & \text{if } v \in [\ell_i, u_i], \\ -100, & \text{if } v \notin [\ell_i, u_i]. \end{cases} \tag{8}$$

Probability tokens outside the valid interval are heavily penalized, while valid tokens are left unchanged. If the API does not support logit bias, or parsing fails, the system falls back to a deterministic mechanism:

$$\hat{p}_i = \text{clip}(p_{raw}, \ell_i, u_i). \tag{9}$$

As a result, the final output always lies within the probability interval derived from the consistency rule.

Algorithm 1 HybridACD Inference Pipeline

Require: Consistency set $T = \{(Q_1, \text{key}_1), \ldots, (Q_k, \text{key}_k)\}$, rule $\mathcal{L}$
Ensure: Predictions $\{\hat{p}_1, \ldots, \hat{p}_k\}$ satisfying $\mathcal{C}(p_1, \ldots, p_k)$

$\mathcal{H} \leftarrow \{\}$ ▷ History of previous predictions
for $i = 1, \ldots, k$ **do**
  $Q_i' \leftarrow \mathcal{M}_{\text{adv}}(Q_i; \tau = 0.7)$ ▷ Adversarial rewriting
  $[\ell_i, u_i] \leftarrow \phi(\mathcal{H}, \text{key}_i, \mathcal{L})$ ▷ Compute bounds
  $\text{cot}_i \leftarrow \mathcal{M}(Q_i')$ ▷ Generate Chain-of-Thought
  $B \leftarrow \{t_v \mapsto b_{t_v}\}_{v \in \mathcal{V}_{\text{num}}}$ ▷ Build logit bias map
  $\hat{p}_i \leftarrow \text{parse}(\text{cot}_i \mid B)$ ▷ Constrained TCD extraction
  $\hat{p}_i \leftarrow \text{clip}(\hat{p}_i, \ell_i, u_i)$ ▷ Deterministic fallback
  $\mathcal{H}[\text{key}_i] \leftarrow \hat{p}_i$ ▷ Record for subsequent bounds
end for
return $\{\hat{p}_1, \ldots, \hat{p}_k\}$

4 Experimental Results

4.1 Experimental setup

HybridACD is evaluated on a forecasting benchmark composed of logically related question sets. The models tested include Gemini-2.5-Flash, GPT-4o-mini, GPT-5.4-mini, MiniMax-M3, Mistral-Medium-3.5-128B, and Mistral-Small-4-119B. For each model, the report compares the BasicForecaster baseline against the full HybridACD variant.

The main metrics are: (i) the Aggregate Violation Score (AVS, lower is better), which measures the degree of consistency violation via an arbitrage measure; (ii) a frequency index, which measures the deviation from the expected distribution; (iii) the Brier Score, which measures actual forecasting error; and (iv) the Calibration Error, which measures the deviation between predicted confidence and empirical accuracy.

4.2 Reduction in consistency violations

Table 2 shows that HybridACD reduces AVS across all six models. The average reduction reaches 95.5%, from 0.1522 down to 0.0053. This indicates that decoding-time constraints can effectively control logical violations without depending strongly on the underlying model architecture.

The largest improvements appear for MiniMax-M3 (-99.6%) and GPT-5.4-mini (-97.8%). For GPT-5.4-mini, the baseline AVS is considerably higher than for the other models, but after HybridACD it drops to 0.0107. This result suggests that TCD is especially useful when a model tends to produce extreme or unstable probabilities on logically related questions.

Table 2 Aggregate Violation Score (AVS, ↓) and Frequency Index (↓) for BasicForecaster versus HybridACD on the Scraped benchmark.

Model	AVS (default)		Frequency		Improvement	
	Basic Forecaster	HybridACD	Basic Forecaster	HybridACD	ΔAVS	Δ%
Gemini-2.5-Flash	0.1116	**0.0087**	0.2612	0.0168	−0.1029	−92.2%
GPT-4o-mini	0.0307	**0.0007**	0.1752	0.0055	−0.0300	−97.6%
GPT-5.4-mini	0.4909	**0.0107**	1.0423	0.0227	−0.4802	−97.8%
MiniMax-M3	0.1266	**0.0005**	0.3756	0.0048	−0.1261	−99.6%
Mistral-Medium-3.5	0.0792	**0.0087**	0.2494	0.0168	−0.0705	−89.1%
Mistral-Small-4	0.0740	**0.0023**	0.2421	0.0156	−0.0717	−96.8%
Average	0.1522	**0.0053**	0.2910	0.0137	−0.1469	−95.5%

4.3 Forecasting accuracy

Table 3 presents the Brier Score on 242 real-world questions. HybridACD improves the Brier Score across all five fully evaluated models, with reductions ranging from 2.4% to 17.3%.

Table 3 Brier Score (BS, ↓) and Calibration Error (CE, ↓) on 242 real-world questions. HybridACD improves BS across all models, confirming the absence of Goodhart's law collapse. † denotes cases where the increase in Calibration Error is still under investigation.

Model	Brier Score		Calibration Error		ΔBS	Goodhart?
	Basic Forcaster	HybridACD	Basic Forcaster	HybridACD		
Gemini-2.5-Flash	0.141	**0.130**	0.195	0.185	−7.8%	No
GPT-4o-mini	0.205	**0.200**	0.161	**0.105**	−2.4%	No
MiniMax-M3	0.123	**0.116**	0.161	**0.117**	−5.7%	No
Mistral-Medium-3.5	0.202	**0.167**	0.090	0.138†	−17.3%	No
Mistral-Small-4	0.211	**0.202**	0.124	0.127†	−4.3%	No

These results indicate that HybridACD does not fall into the Goodhart phenomenon. The reason is that the system does not optimize model weights directly against the consistency metric, but only constrains the final output probability according to mathematical rules. However, the Calibration Error increases for the two Mistral models, suggesting that hard constraints can compress the model's confidence distribution toward the boundary. This is a limitation that should be addressed with softer forms of penalty in future work.

4.4 Comparison with baselines

Table 4 places HybridACD alongside popular forecasting baselines. Compared with ArbitrageForecaster, HybridACD achieves a substantially lower AVS while the cost

per question is much smaller. Notably, HybridACD + MiniMax-M3 attains a Brier Score of 0.116, outperforming more expensive CoT configurations in the comparison table.

Three observations can be drawn. First, HybridACD improves consistency more strongly than post-hoc calibration methods while requiring no repeated optimization rounds. Second, forecasting accuracy is not traded off for consistency. Third, the additional cost of HybridACD mainly comes from the adversarial rewriting step and probability extraction, making it more suitable for real-world deployment than methods that require many iterative rounds.

Table 4 Comprehensive comparison of forecasting methods on the Scraped benchmark. HybridACD achieves the best AVS at negligible cost and with no Goodhart risk. Costs are estimated using June 2026 API pricing.

Method	AVS (↓)	Brier Score (↓)	Cost/Question (USD)	Goodhart
BasicForecaster (GPT-4o-mini)	~ 0.412	0.205–0.231	\$0.002	—
CoT-Forecaster (GPT-4o)	~ 0.287	~ 0.198	\$0.018	—
CoT-Forecaster (o1-preview, best)	~ 0.089	~ 0.170	\$0.05+	—
ArbitrageForecaster ($N = 4$)	~ 0.161	~ 0.245	∼\$2,500	Overfit
HybridACD + GPT-4o-mini	**0.0007**	0.200	\$0.019	No
HybridACD + Gemini-2.5-Flash	0.0087	0.130	∼\$0.07	No
HybridACD + MiniMax-M3	**0.0005**	**0.116**	∼\$0.08	No
HybridACD + Mistral-Medium-3.5	0.0087	0.167	\$0.115	No

4.5 Ablation study

Table 5 evaluates the role of each component in HybridACD. Removing TCD increases AVS from 0.0007 to 0.0147, showing that TCD is the primary component responsible for the consistency guarantee. Removing the adversarial agent increases AVS to 0.0012 and slightly lowers the Brier Score, showing that adversarial rewriting helps improve robustness against linguistic artifacts.

Table 5 Ablation study on GPT-4o-mini. Full HybridACD achieves the best consistency, while each module contributes independently.

Configuration	AVS	Brier Score	Hard Guarantee?
BasicForecaster	0.0307	0.205	No
HybridACD$^{-\text{adv}}$ (TCD only)	0.0012	0.202	Yes
HybridACD$^{-\text{tcd}}$ (Adversarial only)	0.0147	0.206	No
Full HybridACD	**0.0007**	**0.200**	Yes

5 Discussion

The experimental results show that HybridACD achieves its goal of balancing three factors: probabilistic consistency, forecasting accuracy, and inference cost. Methodologically, the key distinguishing feature of HybridACD is the point of intervention. Rather than correcting probabilities after the model has already answered, or updating weights during training, the system intervenes at the final probability-extraction step. As a result, the model is still allowed to reason freely, but the numerical output must lie within the valid domain.

The adversarial component serves as a robustness check against semantics-preserving linguistic transformations. This is especially important for rules such as NEGATION and PARAPHRASE, where the model is prone to being influenced by surface lexical cues. TCD, meanwhile, acts as a mathematical safeguard, ensuring that the final probability does not violate the bounds derived from the consistency rule.

However, HybridACD still has several limitations. First, hard constraints can increase calibration error for some models, particularly when the interval $[\ell, u]$ is too narrow relative to the model's natural distribution. Second, the sequential processing reduces the potential for parallelization on large consistency sets. Third, discretizing probability in steps of 0.01 is suitable for many real-world forecasting problems, but may not be fine-grained enough for applications requiring higher precision.

6 Conclusion

This report has presented HybridACD, a hybrid architecture for improving the probabilistic consistency of LLM forecasters through adversarial rewriting and token-constrained decoding. Building on research into consistency checks, probability-consistent optimization, self-consistency, and constrained decoding, HybridACD chooses the decoding-time point of intervention to avoid the two main limitations of prior approaches: the cost of post-hoc calibration and the Goodhart risk of directly optimizing the model.

Experiments show that HybridACD reduces the mean AVS by 95.5% and improves the Brier Score across the fully evaluated models. This suggests that probabilistic consistency need not be traded off against forecasting accuracy, provided the constraint is applied at the right point. In the future, the system could be extended in three directions: using soft logit bias instead of hard clipping, improving the handling of deeply nested conditional chains, and evaluating on LLM forecasters integrated with RAG or real-time data.

Acknowledgements

References

[1] Paleka, D., Sudhir, A.P., Alvarez, A., Bhat, V., Shen, A., Wang, E., Tramèr, F.: Consistency Checks for Language Model Forecasters (2025). https://arxiv.org/abs/2412.18544

[2] Lowin, J.: An intuitive guide to how llms work. Medium / Personal Blog (2024)

[3] Love, B.C.: Backwards compatible: The strange math behind word order in ai. Cognitive Science Blog (2026)

[4] Anonymous: Probability consistency in large language models: Theoretical foundations meet empirical discrepancies. Submitted to Transactions on Machine Learning Research (Rejected) (2025)

[5] Yang, Y., Ren, H., Lu, Z., Wang, K., Shi, W., Zhou, A., Pan, J., Zhan, M., Li, H.: Probability-consistent preference optimization for enhanced LLM reasoning. In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 6435–6448 (2025). https://aclanthology.org/2025.findings-acl.333/

[6] Zhou, Z., Yuhao, T., Li, Z., Yao, Y., Guo, L.-Z., Ma, X., Li, Y.-F.: Bridging Internal Probability and Self-Consistency for Effective and Efficient LLM Reasoning (2025). https://arxiv.org/abs/2502.00511

[7] Koo, T., Liu, F., He, L.: Automata-based constraints for language model decoding. In: First Conference on Language Modeling (2024). https://openreview.net/forum?id=BDBdblmyzY

[8] Yao, J.-M., Chen, H.-Y., Tang, Z.-X., Tan, B.-J., Peng, S.-W., Xie, B.-C., Su, S.-F.: Token Constraint Decoding Improves Robustness on Question Answering for Large Language Models (2025). https://arxiv.org/abs/2506.09408

[9] Huang, J.Y., Sengupta, S., Bonadiman, D., Lai, Y.-A., Gupta, A., Pappas, N., Mansour, S., Kirchhoff, K., Roth, D.: Deal: Decoding-time alignment for large language models. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, pp. 26280–26300 (2025). https://doi.org/10.18653/v1/2025.acl-long.1274

[10] Schall, M., Melo, G.: The hidden cost of structure: How constrained decoding affects language model performance. In: Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing, pp. 1074–1084 (2025). https://aclanthology.org/2025.ranlp-1.124/

[11] Mousavi, J., Termehchy, A.: Towards consistent language models using controlled prompting and decoding. arXiv preprint (2024)